Question 1 - Ansible configuration management

Question 1: Configure infrastructure and automate application configuration using Ansible by writing playbooks and applying them to target nodes.

All commands below were run on the lab machine and their output is copied verbatim from the recorded logs in `evidence/logs/`. Repository: https://github.com/ksamirdev/ensp461-devops-lab3.

Setup

Two managed nodes are bare Ubuntu 24.04 containers running systemd (`lab/node.Dockerfile`). They start with no Python, nginx or application. Ansible reaches them through the `community.docker.docker` connection plugin, so no SSH keys are needed.

```
$ docker ps --filter name=web-node --format '{{.Names}}  {{.Image}}  {{.Status}}  {{.Ports}}'
web-node-2  ensp461-lab3-node  Up 5 seconds  127.0.0.1:8302->80/tcp
web-node-1  ensp461-lab3-node  Up 5 seconds  127.0.0.1:8301->80/tcp
$ cat automation/inventory.ini
[webnodes]
web-node-1 ansible_connection=community.docker.docker
web-node-2 ansible_connection=community.docker.docker

[webnodes:vars]
ansible_python_interpreter=/usr/bin/python3
$ ansible -i automation/inventory.ini webnodes -m raw -a 'cat /etc/os-release | head -1'
web-node-1 | CHANGED | rc=0 >>
PRETTY_NAME="Ubuntu 24.04.5 LTS"

web-node-2 | CHANGED | rc=0 >>
PRETTY_NAME="Ubuntu 24.04.5 LTS"
```

Playbook

automation/site.yml

```
---
# Configures every web node: the app runs as a systemd service behind nginx.
- name: Deploy text-stats on web nodes
  hosts: webnodes
  gather_facts: false
  vars:
    app_dir: /opt/text-stats
    app_release: "3.0.0"
  pre_tasks:
    - name: Bootstrap python so Ansible modules can run
      ansible.builtin.raw: command -v python3 || (apt-get update -qq && apt-get install -y -qq pytho
n3)
      register: bootstrap
      changed_when: "'/usr/bin/python3' not in bootstrap.stdout"
  tasks:
    - name: Install packages
      ansible.builtin.apt:
        name: [python3-venv, python3-packaging, python3-setuptools, nginx]
        update_cache: true
        cache_valid_time: 3600

    - name: Create service account
      ansible.builtin.user:
        name: textstats
```

```
      system: true
      create_home: false

  - name: Copy application code
    ansible.builtin.copy:
      src: "{{ playbook_dir }}/../service"
      dest: "{{ app_dir }}/"
      owner: root
      mode: preserve
    notify: Restart app

  - name: Copy pinned requirements
    ansible.builtin.copy:
      src: "{{ playbook_dir }}/../requirements.txt"
      dest: "{{ app_dir }}/requirements.txt"
      mode: "0644"

  - name: Install dependencies into a virtualenv
    ansible.builtin.pip:
      requirements: "{{ app_dir }}/requirements.txt"
      virtualenv: "{{ app_dir }}/.venv"
      virtualenv_command: python3 -m venv
    register: pip_install
    retries: 3
    delay: 5
    until: pip_install is succeeded
    notify: Restart app

  - name: Install systemd unit
    ansible.builtin.template:
      src: text-stats.service.j2
      dest: /etc/systemd/system/text-stats.service
      mode: "0644"
    notify: Restart app

  - name: Configure nginx reverse proxy
    ansible.builtin.template:
      src: nginx-site.conf.j2
      dest: /etc/nginx/sites-available/default
      mode: "0644"
    notify: Reload nginx

  - name: Enable and start services
    ansible.builtin.systemd_service:
      name: "{{ item }}"
      enabled: true
      state: started
      daemon_reload: true
    loop: [text-stats, nginx]
handlers:
  - name: Restart app
    ansible.builtin.systemd_service:
      name: text-stats
      state: restarted
      daemon_reload: true
  - name: Reload nginx
    ansible.builtin.systemd_service:
      name: nginx
      state: reloaded
post_tasks:
  - name: Flush handlers before verifying
    ansible.builtin.meta: flush_handlers
  - name: Verify the app answers through nginx
    ansible.builtin.uri:
      url: http://localhost/healthz
      return_content: true
```

```
      register: health
      retries: 5
      delay: 2
      until: health.status == 200
    - name: Show health
      ansible.builtin.debug:
        msg: "{{ inventory_hostname }}: {{ health.json }}"
```

The app runs under systemd as an unprivileged `textstats` user, bound to 127.0.0.1:5000. nginx is the only public listener and proxies to it. The templates are in `automation/templates/`.

First run

```
$ ANSIBLE_DEPRECATION_WARNINGS=false ANSIBLE_FORCE_COLOR=0 ansible-playbook -i automation/inventory.
ini automation/site.yml
...
changed: [web-node-1]
changed: [web-node-2]

TASK [Flush handlers before verifying] *****************************************

TASK [Flush handlers before verifying] *****************************************

TASK [Verify the app answers through nginx] *************************************
ok: [web-node-1]
ok: [web-node-2]

TASK [Show health] *************************************************************
ok: [web-node-1] => {
    "msg": "web-node-1: {'healthy': True, 'release': '3.0.0'}"
}
ok: [web-node-2] => {
    "msg": "web-node-2: {'healthy': True, 'release': '3.0.0'}"
}

PLAY RECAP *********************************************************************
web-node-1                 : ok=13   changed=10   unreachable=0    failed=0    skipped=0    rescued=
0    ignored=0
web-node-2                 : ok=13   changed=10   unreachable=0    failed=0    skipped=0    rescued=
0    ignored=0
```

Second run (idempotency) and verification

```
$ ANSIBLE_FORCE_COLOR=0 ansible-playbook -i automation/inventory.ini automation/site.yml | tail -4
PLAY RECAP *********************************************************************
web-node-1                 : ok=11   changed=0    unreachable=0    failed=0    skipped=0    rescued=
0    ignored=0
web-node-2                 : ok=11   changed=0    unreachable=0    failed=0    skipped=0    rescued=
0    ignored=0
$ ansible -i automation/inventory.ini webnodes -m command -a 'systemctl is-active text-stats nginx'
web-node-1 | CHANGED | rc=0 >>
active
active
web-node-2 | CHANGED | rc=0 >>
active
active
$ curl -s http://localhost:8301/healthz
{"healthy":true,"release":"3.0.0"}
$ curl -s http://localhost:8302/stats -H 'Content-Type: application/json' -d '{"text": "configured b
y ansible configured twice"}'
{"characters":38,"unique_words":4,"words":5}
```

Observations

- First run: **changed=10** per node. Second run with no edits: **changed=0**. Ansible only changes what differs from the declared state, so re-running it is safe.
- Handlers restart the app and reload nginx only when their inputs change. On the second run neither handler fired.
- A `raw` task bootstraps Python first, because every other module needs Python on the node.
- Issues I hit and fixed: `become` failed because the node has no sudo (the docker connection already runs as root). The `pip` module needs `python3-packaging` on the node. A flaky network broke one pip download, so the task now retries 3 times.